Chapter 3: Basing Software Development on Reusable Technology

3.0 Building on the Experience of Others

Why Build X When Y Exists?

Types of reuse:

- Expertise
- Standard designs and algorithms
- Libraries of classes or procedures
- Powerful commands built into languages and operating systems
- Frameworks
- Complete applications

3.1 Frameworks: Reusable Subsystems

A framework is reusable software that implements a generic solution to a generalized problem

Principle: Applications that do different (but related) things tend to have similar designs

3.1.0 Frameworks to promote reuse

A framework is inherently *incomplete*

- Certain classes or methods are used by the framwork but are missing (slots)
- Some functionality is optional, requiring implementation by the dev
- Developers use services provided by the framework. **Services are called the API**

3.1.1 Object-oriented frameworks

In the object oriented paradigm, a framework is composed of a library of classes.

- The API is defined by the set of all public methods of these classes
- Some of the classes will normally be abstract and there are often many Interfaces

3.1.2 Frameworks and product lines

A product line (or product family) is a set of products built on a common base of technology

- The various products in the product line have different features to satisfy different markets
- The software common to all products in included in a framework
- Each product is produced by filling the available hooks and slots

3.1.3 Types of frameworks

- A *horizontal* framework provides general application facilities that a large number of applications can use
- A *vertical* framework (application framework) is more 'complete' but still needs some slots to be filled to adapt it to specific application needs

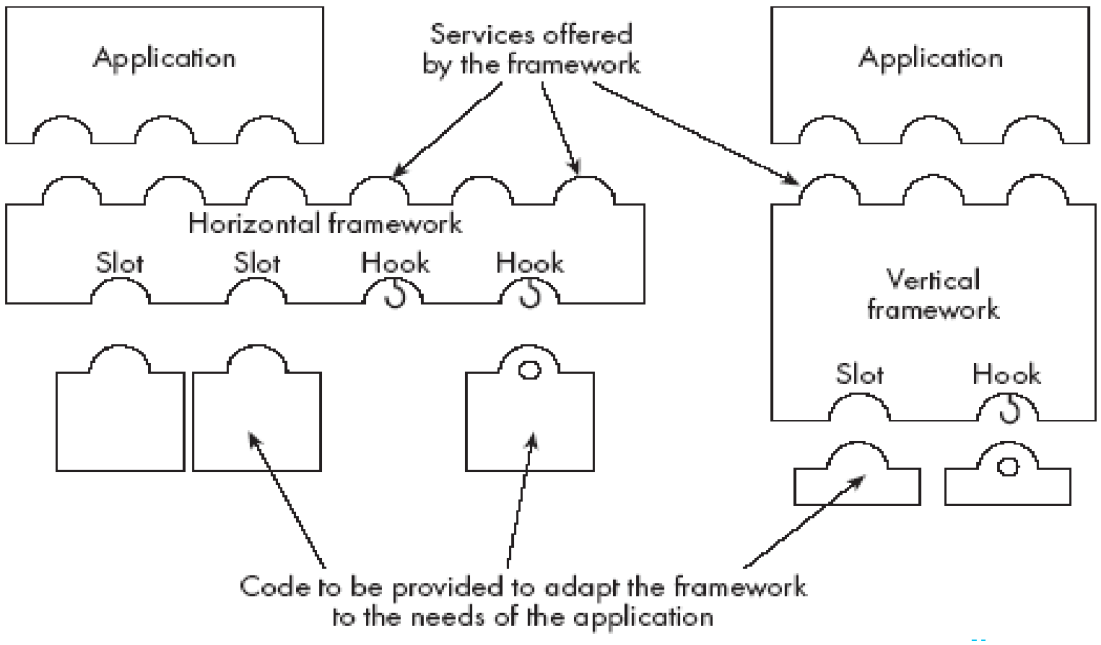


Figure 1: Horizontal vs Vertical Framework

3.2 The Client-Server Architecture

A *Distributed system* is a system in which:

- Computations are performed by separate programs
- processes are ran separate machines
- machines and programs cooperate to perform the task of the system

Server:

- A program that provides a service for other programs that connect to it using a communication channel

Client:

- A program that accesses a server (or multiple) to obtain services
- Multiple clients may access a server simultaneously

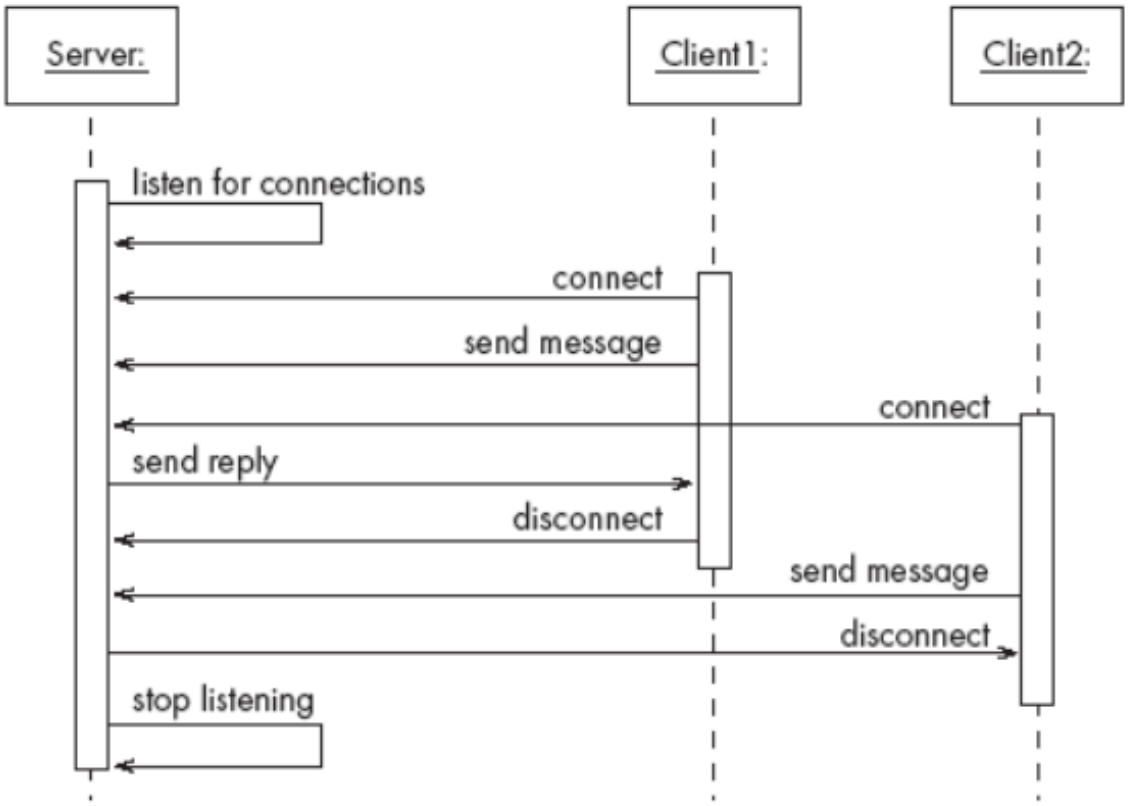


Figure 2: A server program communicated with two client programs

3.2.0 Activities of a server

1. Initializes itself
2. Starts listening for clients
3. Handles the following types of events orginiating from clients
   (a) accepts connections
   (b) responds to messages
   (c) handles client disconnections
4. May stop listening
5. Must cleanly terminate